



Software Requirements Specification – SRS

STUDY MANAGEMENT WEB APP

Table of Contents

Section 1. Overview

1.1 Purpose

1.2 Project Scope

1.3 Definitions

1.4 References

1.5 Overview

Section 2. Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 Users Of The System

2.4 Operating Environment

2.5 Constraints

2.6 Assumptions And Dependencies

2.6.1 Assumptions

2.6.2 Dependencies

2.7 User Needs

3. Specific requirements

3.1 User Authentication

3.2 Manage Tasks

3.3 View Dashboard Progress

3.4 Manage Study Timer

3.5 Use Case Stories

3.6 Use Case Diagram

3.7 Activity Diagrams

4. Non-Functional Requirements

4.1 Performance

4.2 Security

4.3 Usability

4.4 Reliability

- 4.5 Maintainability
- 5. External Interface Requirements
 - 5.1 User Interfaces
 - 5.2 Software Interfaces
 - 5.3 Hardware Interfaces
 - 5.4 Communication Interfaces
- 6. Use Case Summary

Section 1. Overview

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed overview of the Study-Flow web application. It describes the system's functional and non-functional requirements, its architecture, and its external interfaces. This document is intended for developers, project managers, and stakeholders to understand the complete scope of the Study-Flow system.

1.2 Project Scope

Study-Flow is a comprehensive study management web application designed to help students and professionals organize their tasks and track their study time efficiently. The platform features an interactive Kanban board for task management, a Pomodoro-style timer for focused study sessions, and a real-time dashboard for overiewing progress. It is built using React for the frontend and FastAPI for the backend.

1.3 Definitions

Term	Meaning
SPA	Single Page Application.
Kanban	A visual method for managing tasks as they move through various stages (e.g., To Do, In Progress, Done).
Pomodoro Technique	A time management method that uses a timer to break down work into intervals, traditionally 25 minutes in length, separated by short breaks.
JWT	JSON Web Token, used for securely transmitting information between parties as a JSON object.

1.4 References

- React Documentation: <https://reactjs.org/>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- IEEE 830-1998 Standard for Software Requirements Specifications.

1.5 Overview

The remainder of this document is organized as follows:

- Section 2 provides an overall description of the software, its users, and its environment
- Section 3 details the specific functional requirements, use case stories, use case diagrams, and activity diagrams.

- Section 4

Section 2: Overall Description

2.1 Product Perspective

Study-Flow is a standalone, full-stack web application. The frontend is an SPA communicating via RESTful HTTP requests to the FastAPI backend. Data persistence is handled by an SQLite database through SQLAlchemy ORM.

2.2 Product Function

The main functions of the system are:

Function	Description
User Authentication	The user registers, logs in securely, and updates profile information using JWT.
Manage Tasks	The user creates, edits, deletes, and tracks tasks across different statuses using a Kanban board.
View Dashboard Progress	The user views overall study statistics, completed tasks, and current streaks on a dashboard.
Manage Study Timer	The user starts, pauses, and completes Pomodoro timer sessions to track focused study time.
Persist Local State	The system stores tasks, timer progress, and user data locally and synchronizes with the backend database so that information can be restored after the application restarts.

2.3 Users Of The System

Study-Flow implements user accounts and secure authentication. All authenticated users can access the main functions (Task Management, Pomodoro Timer, Dashboard Progress) to organize their study workflows.

Attribute	Detail
Technical level	Basic to intermediate, able to use web applications and navigate intuitive interfaces
Main actions	Register/login, create and manage tasks on a Kanban board, control the Pomodoro timer, and view progress on the dashboard

Access level	Full access to their own tasks and study data (no administrative privileges required)
Environment knowledge	Knows how to use a standard web browser on desktop or mobile devices

2.4 Operating Environment

- Operating system: Windows, macOS, Linux, iOS, or Android
- Delivery format: web application accessible via standard browsers
- Network: internet connection is required for API communication
- User interface: responsive web user interface
- Input devices: keyboard, mouse, or touch screen
- Storage: local storage for JWT tokens and SQLite database on the server

2.5 Constraints

- The application must be accessible on modern web browsers.
- The frontend state management must be handled by Zustand.
- The drag-and-drop functionality must be implemented using react-dnd.
- The backend must process requests concurrently using FastAPI's async capabilities.
- User passwords must be securely hashed using bcrypt before storage.
- The database schema must be managed using SQLAlchemy ORM.

2.6 Assumptions And Dependencies

2.6.1 Assumptions

- The end-user has a stable internet connection.
- The user's device clock is accurate for timer functionalities.

2.6.2 Dependencies

- The drag-and-drop UI depends on the `react-dnd` library.
- Frontend state management relies on `zustand`.

2.7 User Needs

User Need	Why it matters	How the system handles it
Organize tasks visually	Users need to quickly see what tasks are pending, in progress, or completed to manage their workload effectively	The system provides an interactive Kanban board with drag-and-drop capabilities across different status columns
Maintain focus during study sessions	Users often struggle with distractions and need a structured way to concentrate	The system integrates a Pomodoro timer to enforce structured study and break intervals
Track learning progress over time	Users need motivation and insights into how much time they have spent studying	The Dashboard displays aggregate statistics, completed tasks, and study streaks
Securely access personal data from anywhere	Users want their study data to be private, safe, and accessible across different devices	The app requires secure JWT authentication and persists all data safely to a backend database
Avoid losing temporary state	Users should not lose their timer progress or task updates if they accidentally refresh the page	The frontend utilizes Zustand and local storage to sync and persist application state

3. Specific Functional Requirements

3.1 User Authentication

Use Case Name	User Authentication
XRef	Section 2.2 — Product Functions; Section 2.5 — Constraints
Actor	User
Trigger	The user opens the login/register page to access the system.
Precondition	None (for register); User account exists (for login).

Basic Path	1. The user navigates to the login/register page. 2. The user enters their username and password. 3. The user clicks submit. 4. The system validates the input. 5. The system creates the account or verifies credentials. 6. The system generates a JWT token and stores it locally. 7. The system redirects the user to the Dashboard.
Alternative Paths	If the username already exists during registration, the system shows an error message. If the password is incorrect during login, the system shows an "Invalid credentials" error.
Postcondition	The user is authenticated and redirected to the main application.
Exception Paths	The backend server is unreachable; the system displays a network error.
Other	Passwords must be hashed using bcrypt before being stored.

3.2 Manage Tasks

Use Case Name	Manage Tasks
XRef	Section 2.2 — Product Functions
Actor	User
Trigger	The user opens the Tasks (Kanban) tab to organize their work.
Precondition	The user must be authenticated and logged in.
Basic Path	1. The user navigates to the Tasks board. 2. The system loads existing tasks from the database. 3. The user clicks "Add Task" or drags an existing task to a new column. 4. The system updates the task status locally in Zustand. 5. The system sends an API request to persist the new task state to the database.
Alternative Paths	If the API update fails, the system reverts the local UI state to its previous condition and shows a warning.
Postcondition	The task is successfully created, updated, or moved, and the changes are saved to the backend.
Exception Paths	The user loses internet connection; drag-and-drop actions are disabled or cached locally until reconnected.
Other	The drag-and-drop interaction should be smooth and not block the UI during API calls.

3.3 View Dashboard Progress

Use Case Name	View Dashboard Progress
XRef	Section 2.2 — Product Functions
Actor	User
Trigger	The user navigates to the Dashboard tab.
Precondition	The user must be authenticated.
Basic Path	1. The user selects the Dashboard view. 2. The system fetches the user's study metrics, task completion data, and active streaks from the API. 3. The system renders charts and summary statistics.
Alternative Paths	If the user has no previous data, the system displays empty states encouraging the user to start a timer or create a task.
Postcondition	The user sees an accurate reflection of their study progress.
Exception Paths	API timeout occurs while fetching statistics; the system displays an error message and a retry button.
Other	Dashboard rendering must be fast and data should be cached locally to minimize loading times.

3.4 Manage Study Timer

Use Case Name	Manage Study Timer
XRef	Section 2.2 — Product Functions
Actor	User
Trigger	The user opens the Timer tab to start a Pomodoro session.
Precondition	The user must be authenticated.
Basic Path	1. The user navigates to the Timer page. 2. The user selects a session duration. 3. The user clicks "Start Timer". 4. The system begins the countdown. 5. When the timer reaches zero, the system plays an alert sound. 6. The system logs the completed session to the database.
Alternative Paths	The user clicks "Pause" during the session; the timer stops counting down until resumed. The user clicks "Reset"; the timer resets without logging the session.
Postcondition	A study session is completed and recorded in the database.

Exception Paths	The user closes the tab mid-session; the system optionally warns the user or attempts to save partial progress locally.
Other	The timer must rely on robust timestamp calculations to remain accurate if the browser throttles background tabs.

3.5 Use case stories

UC-01: User Authentication

As a user, I want to register and log in to the application so that my personal study data and tasks are securely saved and accessible only to me.

Acceptance Criteria: The user can successfully create an account. Passwords are encrypted. Invalid login attempts return an appropriate error message.

UC-02: Manage Tasks

As a user, I want to create tasks and drag them across different progress columns so that I can visually track what I need to do and what I have finished.

Acceptance Criteria: Tasks can be created and deleted. Tasks dragged to a new column instantly update their status on the frontend and persist to the backend.

UC-03: View Dashboard Progress

As a user, I want to view a summary of my study statistics on a dashboard so that I can evaluate my productivity over time.

Acceptance Criteria: The dashboard accurately reflects the data from the database, updating whenever a new task is completed or a timer session finishes.

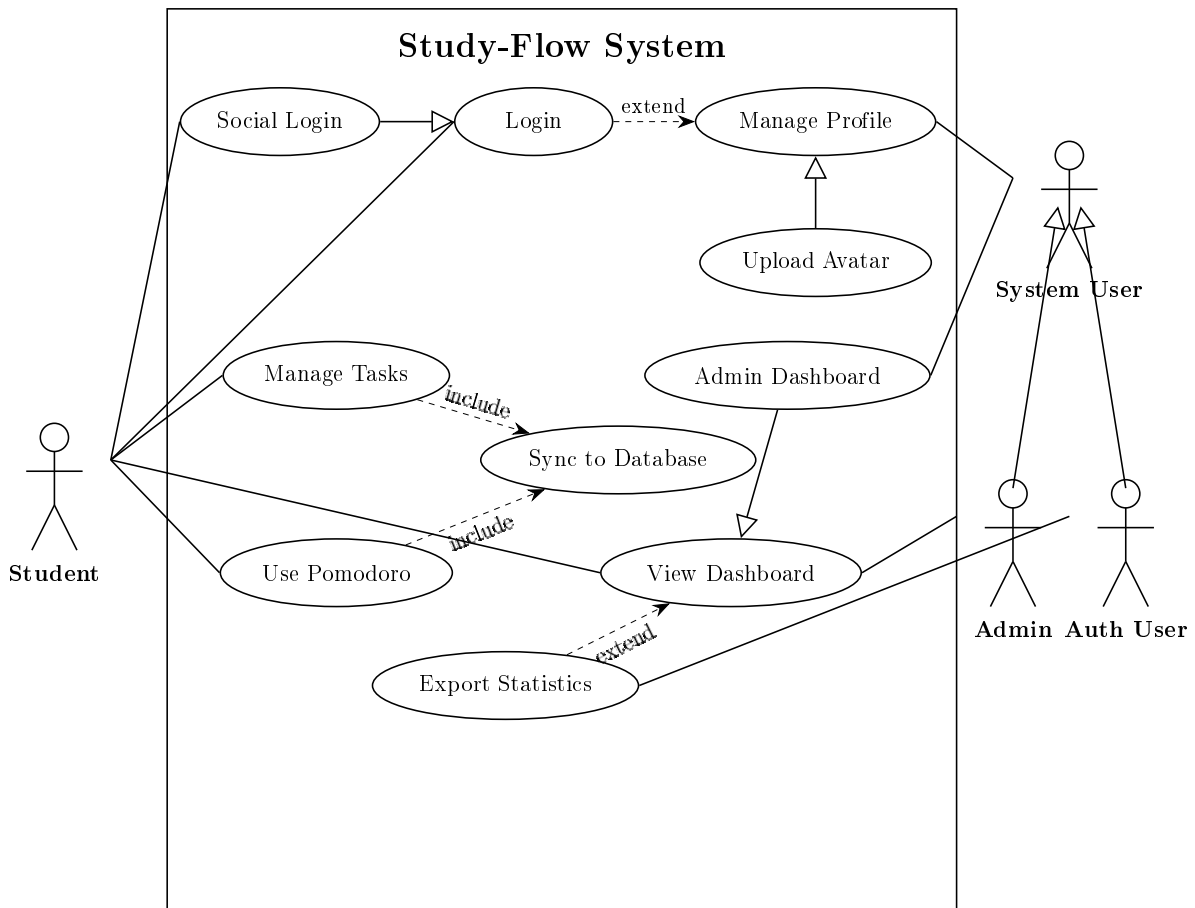
UC-04: Manage Study Timer

As a user, I want to use a Pomodoro timer to manage my study sessions and breaks so that I can maintain focus and prevent burnout.

Acceptance Criteria: The timer counts down accurately. It provides a visual/audio cue upon completion and logs the session.

3.6 Use Case Diagram

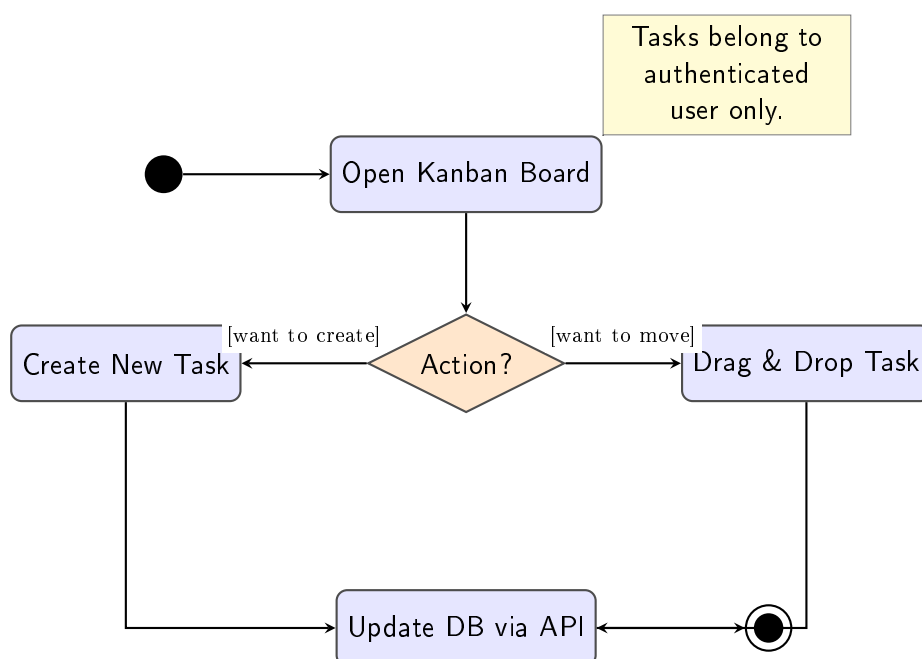
Below is the detailed Use Case Diagram for the Study-Flow system, illustrating actors, system boundaries, and advanced relationships («include», «extend», and generalization).



3.7 Activity Diagram

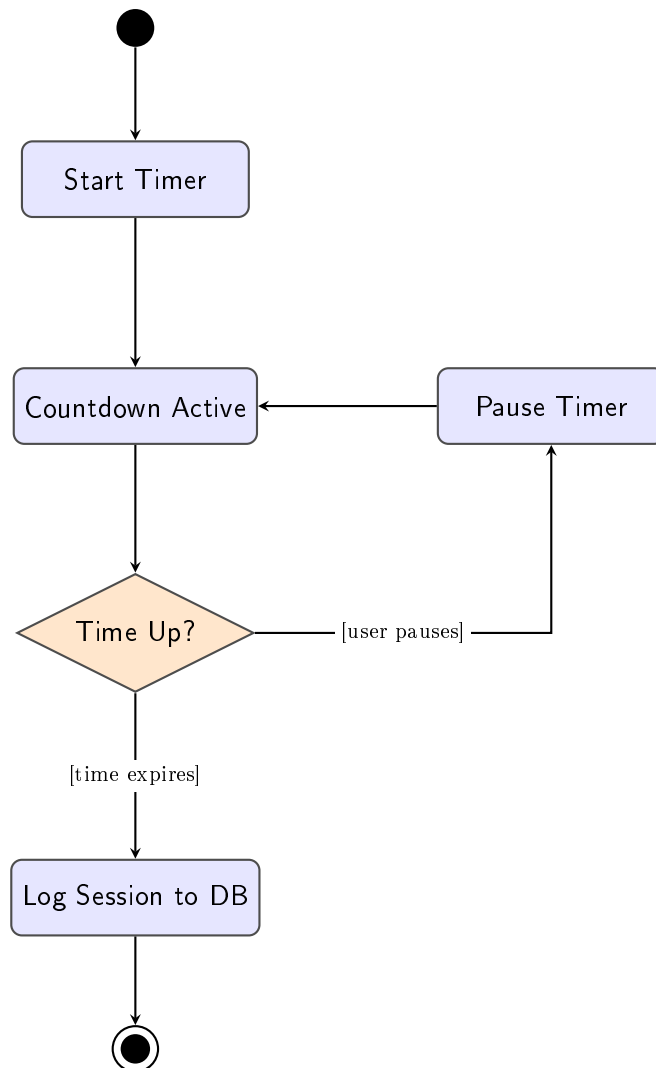
3.7.1 Task Management Flow

This diagram shows the flow from opening the Kanban board to creating or moving tasks and updating the database.



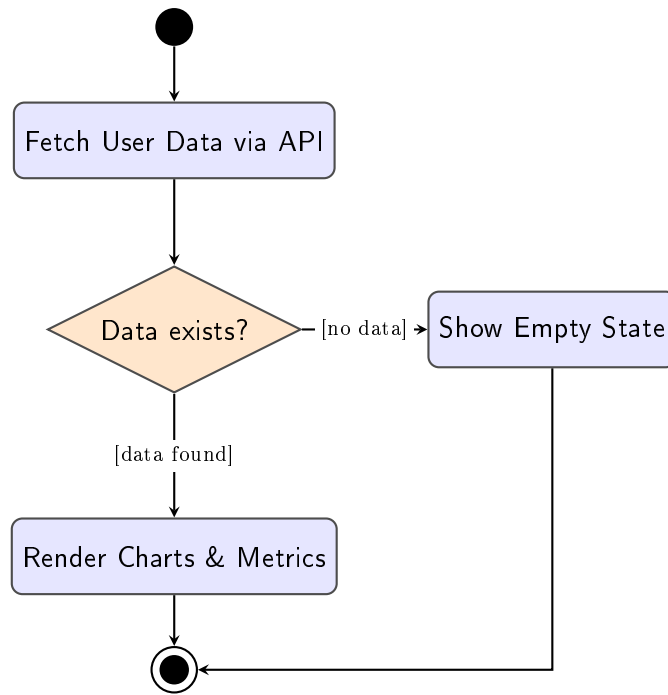
3.7.2 Timer Flow

This diagram shows how the user starts the Pomodoro timer, how pause controls work, and how the session is logged to the database.



3.7.3 Dashboard Progress Flow

This diagram shows how the system fetches user data and statistics via the API and renders charts on the dashboard.



4. Non-Functional Requirements

4.1 Performance

- The Kanban Board and Timer screens should respond to valid input changes (e.g., drag-and-drop actions) within 1 second on a standard laptop/desktop.
- The application should render the Dashboard charts and load user statistics within 2 seconds after successful login, assuming a standard network connection.
- Under default settings, the FastAPI backend should support at least 100 concurrent task synchronization requests smoothly.
- The system should complete API validation and storage of standard CRUD operations (creating tasks, logging sessions) within a reasonable time (under 500ms) for interactive use.
- The application should remain highly responsive while API requests (like task updates) are being processed in the background.
- The system must provide a robust timeout mechanism for API calls (configurable; default 10s) to avoid blocking the user interface if the server is unreachable.
- Client-side state updates (via Zustand) must be idempotent and avoid redundant network requests when local data is already perfectly synced with the server.

4.2 Security

- User passwords must be securely hashed using bcrypt (version 3.2.2 or higher) before being stored in the database.
- The system must use JSON Web Tokens (JWT) for authenticating API requests. Tokens must expire after a configurable timeframe (default 24 hours) and be stored securely on the client side.
- The backend API must implement robust input validation (via Pydantic) to protect against common SQL and NoSQL injection attacks.
- The application must protect against Cross-Site Scripting (XSS) by properly escaping user-generated content before rendering it on the frontend.

4.3 Reliability & Availability

- The system should aim to maintain a high availability rate of 99.9% during normal operating hours.
- In the event of a network failure, the frontend application must gracefully handle offline states, preventing data loss by caching actions (like drag-and-drop or timer completions) locally until the connection is restored.
- The backend API must include comprehensive error handling and logging to prevent server crashes and facilitate rapid debugging.
- Automated daily backups of the SQLite database should be supported to prevent catastrophic data loss.

4.4 Maintainability

- The frontend codebase must be strictly typed using TypeScript, and the backend using Python type hints, to ensure code readability and reduce runtime errors.
- The React application must follow a modular, component-based architecture, ensuring that UI elements (Kanban, Timer, Dashboard) are decoupled and easily testable.
- The backend API endpoints must be fully documented using auto-generated Swagger/OpenAPI documentation natively provided by FastAPI.
- Code must adhere to standard linting and formatting rules (e.g., ESLint/Prettier for React, PEP8/Black for Python).

4.5 Portability & Compatibility

- The web application must be fully responsive and function correctly across major modern web browsers (Google Chrome, Mozilla Firefox, Safari, and Microsoft Edge).
- The frontend UI (built with Tailwind CSS) must support seamless switching between Light and Dark modes based on user preference or system settings.
- The application must be easily deployable on standard cloud infrastructure (AWS, Heroku, Vercel, etc.) using Docker containers if necessary, without requiring vendor-specific modifications.
- The backend API must be OS-agnostic, capable of running smoothly on Windows, Linux, or macOS environments.

5. External Interface Requirements

5.1 User Interfaces

- The user interface must be a web-based application built with React, ensuring a responsive design that adapts to desktop, tablet, and mobile screens.
- The application must feature intuitive drag-and-drop mechanics for task management on the Kanban board.
- The Timer page must offer a clean, distraction-free visual aesthetic to maximize user focus during Pomodoro sessions.
- The Dashboard must present data using clear visual hierarchies and easy-to-read charts for productivity tracking.
- The UI must support standard accessibility guidelines and provide seamless switching between Light and Dark themes via Tailwind CSS.

5.2 Software Interfaces

- **Backend Framework:** The system interfaces with FastAPI (Python) to expose RESTful API endpoints, fully documented via an auto-generated OpenAPI/Swagger UI.
- **Database System:** The application interfaces with a SQLite database for local data storage, abstracted through the SQLAlchemy ORM to allow seamless future migration to PostgreSQL or MySQL.
- **Data Contract:** All data exchanged between the React frontend and the FastAPI backend must be formatted as JSON, with strict validation enforced by Pydantic models.

5.3 Hardware Interfaces

- The application does not require any specialized hardware interfaces. It is designed to run on standard consumer devices, including personal computers, laptops, tablets, and smartphones.
- Interaction with the system requires standard hardware input devices such as a keyboard, mouse, trackpad, or touchscreen display.

5.4 Communication Interfaces

- **Network Protocols:** All client-server communications must utilize standard HTTP/1.1 or HTTP/2 protocols.
- **Security:** In production environments, all communication must be encrypted using HTTPS/TLS to protect sensitive user data (e.g., passwords, JWT tokens) during transit.
- **Data Format:** The system relies exclusively on REST APIs transmitting JSON payloads for data exchange.
- **Connectivity:** An active Internet connection is required for data synchronization, though the frontend local state (Zustand) can temporarily cache actions during brief network interruptions.

6. Use Case Summary

Use Case Name	Actor	Short Description
User Authentication	User	The user registers or logs in to access the system and secure personal data.
Manage Tasks	User	The user creates, updates, and tracks task progress by dragging and dropping them on an interactive Kanban board.
View Dashboard Progress	User	The user views a visual summary of their study statistics, completed tasks, and productivity trends over time.
Manage Study Timer	User	The user runs a Pomodoro timer to manage focus sessions and automatically logs completed study time to the database.